BME 574 — Machine Learning and AI for Biomedical Engineering

Project 5 — Scoring the Unusual

Unsupervised anomaly detection with a variational autoencoder, and whether it was worth building one

Assigned **Monday 9 November** · Due **Monday 23 November, 11:59 pm** · Weight **100 points**

---

1 What you are doing

You will train a variational autoencoder on normal cases only, use it to score cases it has never seen, and then find out whether the exercise was worth it by racing it against three baselines that take about five lines each.

This is the standard recipe for unsupervised anomaly detection, and it appears throughout the biomedical literature: quality control on imaging, novelty detection in physiological monitoring, "flag me the unusual patients" on a clinical register. The recipe is sound. What is often missing from the papers is the control group — the comparison against something trivial.

What is graded is not your AUC. The marks are for a defensible comparison: the model trained only on normal cases, every fitted object fitted only on training rows, the baselines run honestly, the spread across three seeds reported, and the conclusion stated plainly — including when the conclusion is that your model lost.

On at least one of the three tracks the VAE does not beat the trivial baselines. You will not be told which one. Reporting that clearly, with the seed spread to back it up, earns full marks.

2 Your track

You have been assigned one track. The code is identical across all three; only the bundle changes.

	Track A — thyroid	Track B — breast	Track C — retina
what a case is	one patient, 21 assay and demographic features	one fine-needle aspirate, 30 cell-nucleus measurements	one retinal OCT scan, 28×28 grayscale
normal	6,666 euthyroid patients	357 benign aspirates	normal retinas
anomalies	166 hyperthyroid, 368 hypothyroid	212 malignant	choroidal neovascularization, diabetic macular edema, drusen
cases	7,200	569	up to 17,000 after subsampling
source	Quinlan (1987), via PMLB	Street et al. (1993), via scikit-learn	MedMNIST v2 / Kermany et al. (2018)

Tracks A and C have more than one kind of anomaly, and they are not equally detectable. Section 5 requires you to report them separately.

3 The data, and the one rule

Your bundle holds every case, a label, and three index arrays. The split is not an ordinary train/validation/test split:

split	contents	what it is for
train_idx	normal cases only	fitting — the model never sees an anomaly

val_idx	normal and abnormal	choosing anything you need to choose
test_idx	normal and abnormal	scored once, at the end

The rule. Everything that is fitted — the scaler, the PCA, the covariance for the Mahalanobis distance, the one-class SVM, the VAE itself, and the observation variance in TODO 5 — is fitted on train_idx and nothing else. A scaler fitted on all the rows has already told your model where the anomalies are. This is the same lesson as Project 1 and you have now met it four times.

4 What you write — the five TODOs

About thirty lines of PyTorch in total. Every TODO is followed by a check cell; all five must pass.

TODO 1 — the model

An encoder body with two separate heads, one for μ and one for log σ^2, and a decoder that returns to the input width. The check catches an activation on μ and a squashing nonlinearity on the decoder output.

TODO 2 — the sampling step and the loop

The reparameterization trick, $z = \mu + \sigma \odot \varepsilon$, and the five lines of the training step. The check catches returning μ without the noise, which silently turns your VAE into an ordinary autoencoder.

TODO 3 — the loss

The Gaussian reconstruction term with an explicit observation variance σ^2, and the closed-form KL. Both summed over dimensions, not averaged. The check catches the averaging mistake, which over-weights the KL term by a factor of the number of features.

TODO 4 — the anomaly score

The negative ELBO, averaged over several draws of ε. The check verifies that averaging actually reduces the run-to-run wobble.

TODO 5 — the σ^2 you were assuming

Fit the observation variance from the model's own residuals on the training normals, retrain with it, and score again. Report the factor by which the naive loss had been mis-weighting the two terms.

Written for you: the loader, the scaler, the three baselines, the training curve, the ROC figure, the per-class figure, and the verdict cell.

5 What you must compare against

Three baselines are written for you. They are not decoration; the project is about them.

baseline	what it assumes about normal data
Mahalanobis distance	it is one Gaussian blob
PCA reconstruction error, same latent size	it lies near a linear subspace
one-class SVM, RBF kernel	it fills some compact region
supervised logistic regression on the validation split	nothing — it is given the labels, and is there to show what they would have bought

A VAE should beat the first three when the normal data is genuinely curved, clustered or multi-modal. When it does not, that is a finding about your data, not a failed experiment, and it belongs in the memo.

Report per anomaly class. A single pooled AUC averages over whatever mix of abnormal cases happens to be in your test set. If one kind is easy and another is nearly invisible, the pooled number hides it — and in a clinical setting the invisible one is usually the one you cared about.

6 Running it

Everything runs on a laptop CPU. No step in the notebook takes more than about two minutes, and the whole thing finishes in well under ten. You need numpy, torch, scikit-learn and matplotlib — the same stack as Projects 3 and 4.

Work through the preparation notebook first. It is read-and-run, takes about twenty-five minutes, and it is where the ideas are explained; the starter assumes you have done it.

Three mistakes that do not raise an exception. Every one of these produces a model that trains, a loss that falls, and a plausible-looking number. The checks catch all three, but it is worth knowing why they matter.

1. Returning the mean instead of a sample in TODO 2. You have built an ordinary autoencoder; the KL term is still computed but nothing is stochastic, and the posterior variance your encoder reports is never used for anything.
2. Averaging rather than summing over features in TODO 3. The reconstruction term is divided by the number of features and the KL term is not, so the KL is over-weighted by that factor. The model drifts toward the prior, stops encoding anything about the case, and the AUC falls toward chance.
3. Fitting anything on the test rows. The usual symptom is a result that is better than everyone else's and cannot be reproduced.

7 What to hand in

1. The starter notebook, run end to end, with all five checks passing and the figures visible.
2. A one-page memo answering the six questions in §8. PDF or Markdown.
3. AI_USE.md — what you used an AI assistant for, specifically. "Used Claude for debugging" earns nothing; "asked for help reading a shape-mismatch traceback in TODO 3, then found the .mean/.sum confusion myself" earns the marks.

Submit as a single zip named <netid>_BME574_P5.zip.

8 The memo — six questions

One page. Answer with the numbers your notebook printed, not with impressions.

1. What did your VAE score, and what did the best two-line baseline score? State the spread across your three seeds and say whether the gap is larger than it.
2. Which anomaly class was hardest to detect, and why might that be? What would the VAE have to fail to reconstruct in order to flag it?
3. What σ^2 did your residuals imply? What was the naive loss assuming instead, and which direction did that push the balance between the two loss terms?
4. The supervised baseline used labels and your VAE did not. Given the gap you measured, when would you still choose the unsupervised model in a real clinical setting?
5. You reported −ELBO, which is an upper bound on the surprise rather than the surprise itself. Does that matter for ranking cases? Does it matter for reporting an absolute number?
6. Your training split contained no anomalies at all. Name one thing that could go wrong in deployment that this design would be structurally unable to warn you about.

9 Grading

component	points
Notebook runs end to end; all five checks pass	25
No leakage — the official split as given, everything fitted on training normals only	20
Per-class reporting read, not merely printed	15
Comparison done honestly — three seeds, gap weighed against the spread	15
Memo: six questions, one page	20
AI_USE.md, specific rather than generic	5
Total	100

Accuracy and AUC are deliberately not graded. A track where the VAE loses is worth exactly as many marks as one where it wins.

10 References

- Kingma, D. P. & Welling, M. (2014). Auto-Encoding Variational Bayes. ICLR. arXiv:1312.6114. — the model, the reparameterization trick, and the closed-form Gaussian KL used in TODO 3 (appendix B).
- Kingma, D. P. & Welling, M. (2019). An Introduction to Variational Autoencoders. Foundations and Trends in Machine Learning 12(4):307–392. arXiv:1906.02691. — the lecture notation; §2.2 for the ELBO and §2.8 for what goes wrong.
- An, J. & Cho, S. (2015). Variational Autoencoder based Anomaly Detection using Reconstruction Probability. SNU Data Mining Center, Special Lecture on IE, 2015-2. — where this recipe comes from. Note that it is a technical report and was never peer reviewed, which is worth knowing given how heavily it is cited.
- Ruff, L. et al. (2021). A Unifying Review of Deep and Shallow Anomaly Detection. Proceedings of the IEEE 109(5):756–795. — the survey; read §VIII on evaluation before you write the memo.
- Meissen, F., Kaissis, G. & Rueckert, D. (2022). Challenging Current Semi-Supervised Anomaly Segmentation Methods for Brain MRI. BrainLes 2021, LNCS 12962:63–74. — a simple intensity threshold matching or beating learned methods on brain MRI.
- Campos, G. O. et al. (2016). On the evaluation of unsupervised outlier detection. Data Mining and Knowledge Discovery 30:891–927. — why benchmark numbers in this area are so hard to compare.
- Quinlan, J. R. (1987). Simplifying decision trees. International Journal of Man-Machine Studies 27(3):221–234. — the thyroid data. Track A reaches it through PMLB: Romano, J. D. et al. (2022). Bioinformatics 38(3):878–880.
- Street, W. N., Wolberg, W. H. & Mangasarian, O. L. (1993). Nuclear feature extraction for breast tumor diagnosis. IS&T/SPIE 1905:861–870. — Track B.
- Yang, J. et al. (2023). MedMNIST v2. Scientific Data 10:41; source data Kermany, D. S. et al. (2018). Cell 172(5):1122–1131. — Track C.